FANTASYREALM

Documentation technique de l'application

SOMMAIRE

1. PRÉSENTATION TECHNIQUE 3
   1.1. Objectifs et périmètre 3
   1.2. Choix technologiques 3
   1.3. Architecture générale 4
   1.4. Organisation du code 4
2. FONCTIONNEMENT DU FRONTEND 6
   2.1. Structure des interfaces 6
   2.2. Communication avec les API 6
   2.3. Moteur de création des personnages 7
   2.4. Personnalisation et équipements 9
3. FONCTIONNEMENT DU BACKEND 10
   3.1. Organisation des API PHP 10
   3.2. Authentification et rôles 11
   3.3. Gestion des personnages 12
   3.4. Modération et administration 13
   3.5. Notifications électroniques et contact 14
4. GESTION DES DONNÉES 15
   4.1. Base relationnelle MariaDB 15
   4.2. Modèle conceptuel et modèle logique 16
   4.3. Contraintes d'intégrité 17
   4.4. Base documentaire MongoDB 17
   4.5. Cohérence entre MariaDB et MongoDB 19
5. SÉCURITÉ DE L'APPLICATION 20
   5.1. Politique de mots de passe 20
   5.2. Sessions et CSRF 20
   5.3. Autorisations et contrôle d'accès 21
   5.4. Validation des données 21
   5.5. Configuration et infrastructure 21
6. DIAGRAMMES TECHNIQUES 22
   6.1. Modèle conceptuel de données (MCD) 22

6.2. Diagramme de cas d'utilisation UML 23
6.3. Diagramme de séquence UML — Création d'un personnage 24
6.4. Arborescence du site 25
7. TESTS, MAINTENANCE ET ÉVOLUTIONS 25
7.1. Tests fonctionnels 25
7.2. Tests techniques 26
7.3. Maintenance 27
7.4. Évolutions possibles 27
CONCLUSION 27

1. PRÉSENTATION TECHNIQUE

1.1. Objectifs et périmètre

FantasyRealm est une application web destinée à compléter l'expérience du MMORPG FantasyRealm Online développé par PixelVerse Studios.

Elle permet aux joueurs de créer et de personnaliser des personnages fantastiques, de gérer leurs équipements et de partager leurs créations avec la communauté.

L'application comprend également un système de modération et des espaces d'administration adaptés aux différents rôles.

La solution repose sur une architecture client-serveur composée de trois ensembles principaux :

- Le frontend, qui assure l'affichage et les interactions.
- Le backend, qui exécute les traitements métier et contrôle les autorisations.
- Les systèmes de stockage, qui conservent les données applicatives et les historiques.

L'ensemble est exécuté dans un environnement conteneurisé avec Docker.

1.2. Choix technologiques

Technologie	Utilisation et justification
HTML5	Structuration sémantique des pages
CSS3	Mise en forme, adaptation responsive et identité visuelle
JavaScript natif	Interactions, affichage dynamique et communication avec l'API
Canvas API	Composition et coloration des personnages
PHP 8.5	Traitements métier, sécurité et API
Apache	Distribution des ressources et exécution du backend
MariaDB 11.4	Stockage relationnel des données applicatives
MongoDB 7.0	Journalisation et historiques sous forme de documents
PDO	Accès à MariaDB par requêtes préparées
Composer	Gestion des bibliothèques PHP

PHPMailer	Envoi des notifications SMTP
Docker Compose	Configuration et exécution de l'environnement
Git et GitHub	Versionnement et suivi du code source

Le choix de JavaScript natif permet de réaliser les interactions nécessaires sans recourir à un framework frontend.

PHP centralise les traitements et les contrôles de sécurité. MariaDB assure l'intégrité des données structurées, tandis que MongoDB permet de conserver des événements dont la structure est adaptée à l'historisation.

1.3. Architecture générale

Le navigateur charge les pages HTML, les styles CSS et les scripts JavaScript depuis Apache.

Les interactions nécessitant un traitement serveur déclenchent des requêtes HTTP vers les API PHP.

Ces API valident les données, appliquent les règles métier et consultent les bases de données.

Les réponses sont généralement retournées au format JSON.

L'application utilise une authentification par session PHP. Les requêtes du navigateur transmettent le cookie de session, tandis que les opérations sensibles utilisent un jeton CSRF.

Le backend accède à MariaDB par PDO et à MongoDB par le client PHP MongoDB.

Les notifications électroniques sont transmises par PHPMailer au serveur SMTP configuré.

1.4. Organisation du code

L'arborescence principale du projet est la suivante :

```
FantasyRealm/
|
|-- Frontend/
|   |-- Html/
|   |-- Css/
|   |-- js/
|   |   |-- components/
|   |-- assets/
```

```
|       |-- backgrounds/
|       |-- character-creator/
|       |-- accessories/
|       |-- logo/
|
|-- Backend/
|   |-- api/
|   |-- config/
|   |-- Database/
|   |   |-- shema.sql
|   |   |-- seed.sql
|   |-- composer.json
|   |-- composer.lock
|
|-- docker/
|   |-- Dockerfile
|   |-- apache.conf
|
|-- scripts/
|   |-- init-local.sh
|   |-- start-local.sh
|   |-- stop-local.sh
|
|-- compose.yml
|-- compose.local.yml
|-- compose.prod.yml
|-- README.md
```

Le répertoire Frontend/Html contient les pages publiques et les tableaux de bord.

Les feuilles CSS définissent l'identité visuelle et les adaptations responsive.

Les scripts JavaScript prennent en charge les formulaires, les galeries, les tableaux de gestion et le moteur de rendu.

Le répertoire Backend/api regroupe les traitements associés aux fonctionnalités.

Le répertoire Backend/config contient notamment la connexion PDO, la sécurité des sessions et la politique de mots de passe.

Les scripts SQL définissent la structure relationnelle et les données initiales.

Les fichiers Docker sont séparés du code applicatif afin de faciliter la maintenance de l'environnement.

2. FONCTIONNEMENT DU FRONTEND

2.1. Structure des interfaces

Le frontend repose sur plusieurs pages HTML, notamment :

- index.html : page d'accueil.
- character-gallery.html : galerie publique.
- character.html : détail d'un personnage.
- create-character.html : créateur de personnages.
- dashboard.html : espace personnel.
- employee-dashboard.html : gestion employé.
- admin-dashboard.html : administration.
- register.html et login.html : authentification.
- contact.html : formulaire de contact.

Des composants JavaScript partagés assurent la cohérence de certains éléments de navigation, notamment l'en-tête et le pied de page.

Les interfaces sont conçues pour s'adapter aux formats desktop et mobile.

2.2. Communication avec les API

Le frontend utilise la Fetch API pour transmettre et récupérer les données.

Les requêtes de consultation utilisent principalement la méthode GET.

Les opérations de création, de modification ou de modération utilisent des requêtes POST, avec un corps JSON lorsque cela est prévu par l'API.

Une requête de création de personnage peut notamment transmettre les informations suivantes :

```
{
  "name": "Moonwhisper",
```

"gender": "female",

"nose_style": 1,

"mouth_style": 2,

"eye_style": 3,

"hair_style": 4,

"skin_color": "#d8b39a",

"eye_color": "#57b8ff",

"hair_color": "#3e3042"

}

Cet exemple illustre la structure des paramètres utilisés lors de la création.

Le backend contrôle les types, les valeurs autorisées et les contraintes métier avant l'enregistrement.

Les réponses HTTP permettent au frontend d'afficher un message de confirmation ou de présenter une erreur.

Les codes utilisés comprennent notamment 200, 201, 400, 401, 403, 404, 409, 422 et 500, selon les opérations.

2.3. Moteur de création des personnages

La personnalisation graphique constitue l'une des fonctionnalités centrales de FantasyRealm.

Elle repose sur un système de superposition d'images PNG transparentes.

Chaque personnage est composé de plusieurs éléments graphiques qui partagent les mêmes dimensions et coordonnées.

Le moteur utilise une surface Canvas de : **568 × 1 275 pixels.**

Les éléments graphiques disponibles comprennent notamment :

- Un corps masculin ou féminin.
- Dix formes de nez.
- Sept formes de bouche.
- Sept formes d'yeux.
- Dix coiffures.

- Des couleurs personnalisables.
- Des images d'équipements et d'accessoires.

Les ressources sont regroupées dans le répertoire : Frontend/assets/character-creator/

Les sous-dossiers homme et femme contiennent les ressources propres à chaque type de corps.

Le script Frontend/js/create-character.js gère l'interface de création, les choix de caractéristiques, la coloration et l'aperçu visuel.

Le composant Frontend/js/components/character-renderer.js fournit un moteur partagé permettant de reconstituer les personnages enregistrés.

Ce moteur est utilisé notamment pour l'affichage des personnages dans les tableaux de bord, la galerie et les pages de détail.

Principe de composition

Les ressources graphiques sont chargées et superposées dans un ordre défini.

Le moteur partagé prend notamment en charge :

1. Les vêtements ou armures.
2. Le corps de remplacement, lorsqu'un équipement en fournit un.
3. Le nez et la coloration de la peau.
4. La bouche.
5. Les yeux et la coloration des iris.
6. Les cheveux et leur coloration.
7. Les armes.
8. Les accessoires.

L'ordre de superposition est important pour éviter qu'un élément soit affiché derrière une partie qui devrait être située en dessous.

Gestion des couleurs

La coloration utilise les fonctionnalités de manipulation de pixels et de composition du Canvas.

La peau est teintée en conservant autant que possible les ombres et reliefs de l'image d'origine.

Les couleurs des cheveux sont appliquées sur une copie de leur couche graphique.

La coloration des yeux utilise un masque afin de cibler les iris sans modifier toute l'image.

Chargement des ressources

Le moteur utilise un cache JavaScript pour éviter de recharger inutilement les mêmes images.

Les chemins d'images des équipements sont validés avant leur utilisation.

Le composant vérifie également les valeurs d'apparence et les dimensions attendues des ressources principales.

L'utilisation d'un moteur partagé permet d'obtenir un rendu cohérent entre les différents écrans.

2.4. Personnalisation et équipements

Après validation du nom de son personnage, l'utilisateur peut modifier son apparence et sélectionner des équipements.

Les équipements peuvent appartenir à quatre catégories :

- armor
- clothing
- weapon
- accessory

Les armures et les vêtements sont associés à un type de corps compatible.

Les armes et les accessoires sont traités comme des équipements utilisables indépendamment du genre.

Le backend applique des règles empêchant la sélection de combinaisons invalides.

Un personnage peut porter au maximum un élément de type armure ou vêtement et une arme.

Plusieurs accessoires peuvent être associés au même personnage.

Les équipements disponibles sont enregistrés dans MariaDB et leurs fichiers graphiques sont conservés dans le répertoire prévu pour les ressources téléversées.

Un volume Docker persistant permet de préserver ces fichiers.

3. FONCTIONNEMENT DU BACKEND

3.1. Organisation des API PHP

Le backend est organisé autour de fichiers PHP spécialisés.

Chaque API traite une ou plusieurs opérations liées à une fonctionnalité.

Domaine	Principaux fichiers
Authentification	register.php, login.php, logout.php, session.php
Mot de passe	forgot-password.php, reset-password.php
Personnages	characters.php, character.php, edit-character.php
Gestion personnelle	my-characters.php, duplicate-character.php, share-character.php
Suppression	delete-character.php, restore-character.php, permanent-delete-character.php
Galerie	gallery.php, character-equipment-data.php
Avis	reviews.php, submit-review.php, moderate-review.php
Modération	pending-characters.php, moderate-character.php
Équipements	accessories.php, add-accessory.php, delete-accessory.php
Gestion employé	employee-users.php, employee-characters.php, employee-contact-messages.php
Administration	admin-employees.php, admin-create-employee.php, admin-activity-logs.php
Historique	activity-logger.php, character-history-logger.php, admin-character-history.php
Contact	contact.php

Cette organisation permet de séparer les responsabilités des différents traitements.

Chaque endpoint valide les données reçues et retourne une réponse adaptée.

Les contrôles d'identité et de permissions sont effectués côté serveur.

3.2. Authentification et rôles

L'application utilise trois rôles de comptes :

USER

Accès aux fonctionnalités de création, personnalisation, gestion et partage des personnages.

EMPLOYEE

Accès aux opérations de modération et de gestion autorisées.

ADMIN

Accès aux fonctionnalités administratives, à la gestion des employés et à la consultation des historiques.

Inscription

L'inscription est traitée par register.php.

Le backend contrôle les informations saisies et vérifie l'unicité de l'adresse électronique et du pseudonyme.

Les mots de passe sont validés selon une politique commune définie dans Backend/config/password.php.

Après validation, le mot de passe est haché avant son enregistrement dans MariaDB.

Le compte créé reçoit un rôle utilisateur standard.

Connexion

L'API login.php vérifie l'identité de l'utilisateur.

La vérification du mot de passe utilise password_verify().

Le système comporte un mécanisme de limitation des tentatives de connexion, dont les données sont enregistrées dans la table login_rate_limits.

Après authentification, l'identifiant de session est renouvelé afin de limiter les risques de fixation de session.

Contrôle des permissions

Les endpoints privés vérifient l'existence d'une session active.

Le statut du compte est contrôlé dans MariaDB.

Les comptes suspendus ne peuvent pas utiliser les opérations protégées.

Les fonctions administratives s'appuient sur une vérification explicite du rôle de l'utilisateur.

Les permissions ne reposent donc pas uniquement sur la visibilité des éléments dans les interfaces JavaScript.

3.3. Gestion des personnages

Création

La création est principalement traitée par characters.php.

Le backend vérifie le nom, le genre et les différentes options d'apparence.

Les valeurs numériques des caractéristiques sont contrôlées selon les limites du catalogue graphique.

Les couleurs doivent être des valeurs hexadécimales valides.

Le personnage est ensuite enregistré avec le statut PENDING et la valeur de partage désactivée.

Une transaction SQL permet d'enregistrer ensemble le personnage dans characters et son apparence dans appearances.

Après validation de la transaction, un événement d'historique peut être enregistré dans MongoDB.

Modification

L'API edit-character.php permet de modifier les personnages approuvés appartenant à l'utilisateur.

Elle contrôle la propriété du personnage, son statut et les caractéristiques transmises.

Les équipements sont également vérifiés avant enregistrement.

La modification de l'apparence et le remplacement des associations d'équipements sont réalisés dans une transaction SQL.

L'enregistrement de l'historique MongoDB est requis avant la validation SQL de cette opération.

Duplication

L'API duplicate-character.php récupère les données du personnage source et crée un nouveau personnage.

Une transaction SQL permet de copier son apparence et ses associations d'équipements.

Le nouveau personnage reçoit un nom distinct, le statut PENDING et un partage désactivé.

Il devra donc être validé avant sa publication.

Partage

L'API share-character.php contrôle la propriété du personnage.

Elle autorise le partage uniquement lorsque le personnage est approuvé.

L'arrêt du partage reste possible indépendamment de son état d'approbation.

Suppression

L'application propose des opérations de suppression logique, de restauration et de suppression définitive.

La suppression logique repose sur le champ deleted_at.

Les personnages marqués comme supprimés sont exclus des consultations publiques.

Certaines opérations administratives ou de modération peuvent entraîner une suppression définitive selon les règles de gestion.

3.4. Modération et administration

Modération des noms

Lorsqu'un personnage est créé, son nom est soumis à validation.

L'employé peut approuver ou refuser la demande.

En cas de refus, un motif est obligatoire.

L'API moderate-character.php conserve les informations de décision dans une table dédiée avant la suppression définitive d'un personnage rejeté.

Les opérations SQL nécessaires sont regroupées dans une transaction.

Après la décision, une notification électronique peut être envoyée au propriétaire.

Les événements de modération font également partie des opérations pouvant être journalisées dans MongoDB.

Modération des avis

Un utilisateur peut déposer un avis sur un personnage accessible publiquement.

L'avis est enregistré avec le statut PENDING.

Un employé peut ensuite l'approuver ou le refuser.

Seuls les avis approuvés sont présentés publiquement.

Lorsqu'un avis est approuvé, le système tente d'envoyer une notification au propriétaire du personnage.

Administration

Les administrateurs disposent de fonctionnalités supplémentaires permettant notamment de créer et de gérer les comptes employés.

Ils peuvent également consulter les événements d'activité et les historiques de personnalisation.

Les données sont récupérées à travers des endpoints réservés aux comptes disposant du rôle ADMIN.

3.5. Notifications électroniques et contact

L'application utilise PHPMailer pour transmettre des e-mails par SMTP.

Les paramètres sont chargés depuis la configuration privée de l'application.

Les fonctionnalités concernées comprennent :

- La récupération du mot de passe.
- Les décisions de modération des personnages.
- L'approbation des avis.
- Les demandes de contact.

Récupération du mot de passe

L'utilisateur fournit les informations nécessaires à sa demande.

Le backend génère un jeton aléatoire à durée de validité limitée.

Une version hachée du jeton est conservée dans la base.

Le lien de réinitialisation est envoyé par courrier électronique.

Lorsqu'un nouveau mot de passe est défini, le jeton doit être valide et ne peut plus être utilisé après sa consommation.

Formulaire de contact

Le fichier contact.php reçoit les données du formulaire.

Les informations sont validées côté serveur avant l'enregistrement.

Le message est conservé dans la table contact_messages.

Le backend tente ensuite de transmettre une notification par SMTP à l'adresse configurée avec MAIL_CONTACT_TO.

L'adresse de réponse correspond à celle du demandeur.

L'API renvoie un indicateur email_sent permettant de distinguer l'enregistrement en base de la réussite de la transmission SMTP.

Les messages restent consultables dans le tableau de bord employé, y compris lorsqu'une notification électronique échoue.

L'accès à cette consultation est réservé aux employés et administrateurs actifs.

4. GESTION DES DONNÉES

4.1. Base relationnelle MariaDB

MariaDB constitue le système principal de stockage.

Les données sont organisées dans des tables liées par des clés primaires et des clés étrangères.

Le schéma est défini dans : Backend/Database/shema.sql

Le fichier : Backend/Database/seed.sql permet d'initialiser les rôles nécessaires au fonctionnement de l'application.

Tables principales

Table	Rôle
roles	Définition des rôles
users	Comptes et informations d'authentification

characters	Personnages et état de modération
appearances	Caractéristiques physiques
accessories	Catalogue des équipements
character_accessories	Associations personnages-équipements
reviews	Notes, commentaires et statut de modération
contact_messages	Demandes transmises par les utilisateurs
login_rate_limits	Limitation des tentatives de connexion
character_moderation_decisions	Historique des décisions sur les noms
character_staff_deletions	Traçabilité des suppressions effectuées par le personnel

Les tables de modération et de suppression du personnel permettent de conserver des informations même lorsque le personnage concerné a été supprimé.

4.2. Modèle conceptuel et modèle logique

Principales relations

Un rôle peut être attribué à plusieurs utilisateurs, mais chaque utilisateur possède un rôle.

Un utilisateur peut créer plusieurs personnages.

Chaque personnage possède une apparence enregistrée dans la table appearances.

Un personnage peut être associé à plusieurs accessoires.

Un accessoire peut être associé à plusieurs personnages.

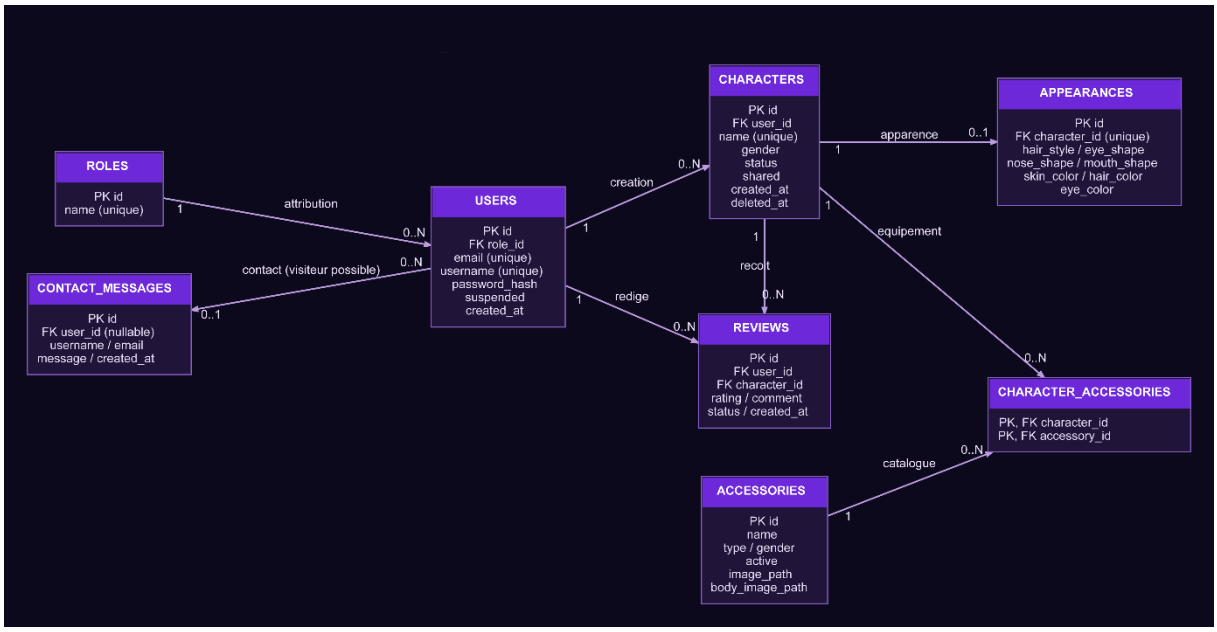
Cette relation plusieurs-à-plusieurs est représentée par la table de liaison character_accessories.

Un utilisateur peut déposer plusieurs avis.

Un personnage peut recevoir plusieurs avis.

Un message de contact peut être associé à un utilisateur inscrit ou provenir d'un visiteur non connecté.

Représentation simplifiée du modèle logique



Le modèle conceptuel de données de FantasyRealm présente les principales entités de la base MariaDB, leurs associations et leurs cardinalités. Le diagramme correspondant est présenté dans la section 6.1 de cette documentation.

4.3. Contraintes d'intégrité

Les contraintes SQL permettent de limiter les incohérences.

L'unicité des adresses électroniques et des pseudonymes est assurée dans users.

L'unicité du nom des personnages est garantie par une contrainte sur characters.name.

La table appearances impose l'unicité de character_id, afin qu'un personnage possède au plus une ligne d'apparence.

La table de liaison character_accessories utilise une clé primaire composée, empêchant l'enregistrement d'une même association deux fois.

Les clés étrangères assurent la cohérence des relations.

Les contraintes fonctionnelles complémentaires, comme les compatibilités d'équipements ou les statuts autorisés, sont également contrôlées par le backend.

4.4. Base documentaire MongoDB

MongoDB est utilisée pour conserver les événements et historiques.

Deux collections principales sont exploitées.

Collection activity_logs

Cette collection conserve les activités effectuées par les employés et administrateurs.

Un document comprend notamment :

```
{
  "_id": "identifiant_evenement",
  "actor_id": 4,
  "action": "character.approved",
  "target_type": "character",
  "target_id": 12,
  "created_at": "date BSON UTC"
}
```

L'exemple ci-dessus est schématique. Dans MongoDB, la date est enregistrée sous forme de valeur BSON et non nécessairement comme une chaîne de caractères.

Les types d'événements sont contrôlés par une liste d'actions autorisées dans activity-logger.php.

Collection character_history

Cette collection conserve des instantanés de personnalisation.

Elle peut notamment enregistrer :

- L'identifiant du personnage.
- Son nom.
- L'identifiant du propriétaire.
- Le genre.
- Les caractéristiques physiques.
- Les équipements associés.
- Le type d'opération.
- La date d'enregistrement.

Les événements pris en charge comprennent :

- character.created
- character.duplicated
- character.updated

Lors d'une duplication, un identifiant de personnage source peut également être conservé.

Ces informations permettent de consulter les différentes personnalisations enregistrées.

4.5. Cohérence entre MariaDB et MongoDB

MariaDB et MongoDB remplissent des fonctions complémentaires, mais leurs écritures ne participent pas à une transaction distribuée unique.

Le comportement dépend des opérations.

Lors de la création et de la duplication d'un personnage, la transaction SQL est validée en premier.

Le backend tente ensuite d'enregistrer l'historique dans MongoDB.

Si cet enregistrement échoue, la création ou la duplication SQL demeure valide, mais une entrée d'historique peut manquer.

Lors de la modification d'un personnage approuvé, le backend tente d'enregistrer l'historique MongoDB avant de valider la transaction SQL.

Une erreur MongoDB entraîne alors l'annulation de la transaction SQL.

Cette approche réduit le risque d'une modification confirmée sans historique, mais ne garantit pas une atomicité globale entre les deux bases. Une défaillance SQL après l'écriture MongoDB pourrait laisser un événement d'historique qui ne correspond pas à une modification validée.

Il sera donc important de prévoir des tests de défaillance et, à terme, d'envisager une stratégie de réconciliation ou de journalisation transactionnelle pour les opérations critiques.

5. SÉCURITÉ DE L'APPLICATION

5.1. Politique de mots de passe

La politique de sécurité des mots de passe est centralisée dans Backend/config/password.php.

Elle impose notamment :

- Au moins douze caractères.
- Une lettre majuscule.
- Une lettre minuscule.
- Un chiffre.
- Un caractère spécial.
- Une longueur maximale de 72 octets.

Les mots de passe sont hachés avec Argon2id.

Les paramètres définis dans le code comprennent une consommation mémoire de 19 456 KiB, un coût temporel de 2 et un thread.

Les mots de passe ne sont jamais conservés en clair dans MariaDB.

5.2. Sessions et CSRF

Le fichier Backend/config/security.php définit la fonction de démarrage sécurisé des sessions.

Les paramètres prévoient notamment :

- session.use_strict_mode
- session.use_only_cookies
- Des cookies HttpOnly.
- Une politique SameSite=Lax.
- Un attribut Secure lorsque la connexion est détectée en HTTPS.

Un jeton CSRF est généré avec random_bytes() et conservé dans la session.

Les requêtes protégées transmettent ce jeton dans l'en-tête X-CSRF-Token.

La vérification utilise hash_equals() afin de comparer les jetons.

Une requête ne disposant pas d'un jeton valide peut être rejetée avec le code HTTP 403.

5.3. Autorisations et contrôle d'accès

Les endpoints privés vérifient l'identité du compte et son état actif.

Les fonctions administratives contrôlent également le rôle actuel enregistré en base.

La propriété des personnages est vérifiée lors des opérations personnelles.

Un utilisateur ne peut donc pas modifier librement un personnage appartenant à un autre compte.

Les endpoints réservés aux employés et aux administrateurs ne doivent pas se fier aux informations de rôle transmises par le navigateur.

Les comptes suspendus sont refusés lors des opérations protégées.

5.4. Validation des données

Les informations reçues par le backend sont contrôlées avant leur utilisation.

Les requêtes SQL utilisent PDO et des paramètres préparés.

Les noms et identifiants sont soumis à des vérifications de format ou de longueur.

Les valeurs d'apparence et les couleurs sont comparées aux formats autorisés.

Les chemins d'images des équipements font également l'objet de contrôles.

Le frontend doit afficher les contenus utilisateurs sans permettre l'exécution de code injecté dans les pages.

L'utilisation de validations côté serveur reste indispensable, même lorsque des contrôles JavaScript sont présents.

5.5. Configuration et infrastructure

Les identifiants MariaDB, MongoDB et SMTP sont conservés dans des fichiers de configuration privés.

Les fichiers .gitignore et .dockerignore contribuent à limiter leur présence dans le dépôt et les images.

L'environnement Docker utilise un réseau privé pour les bases de données.

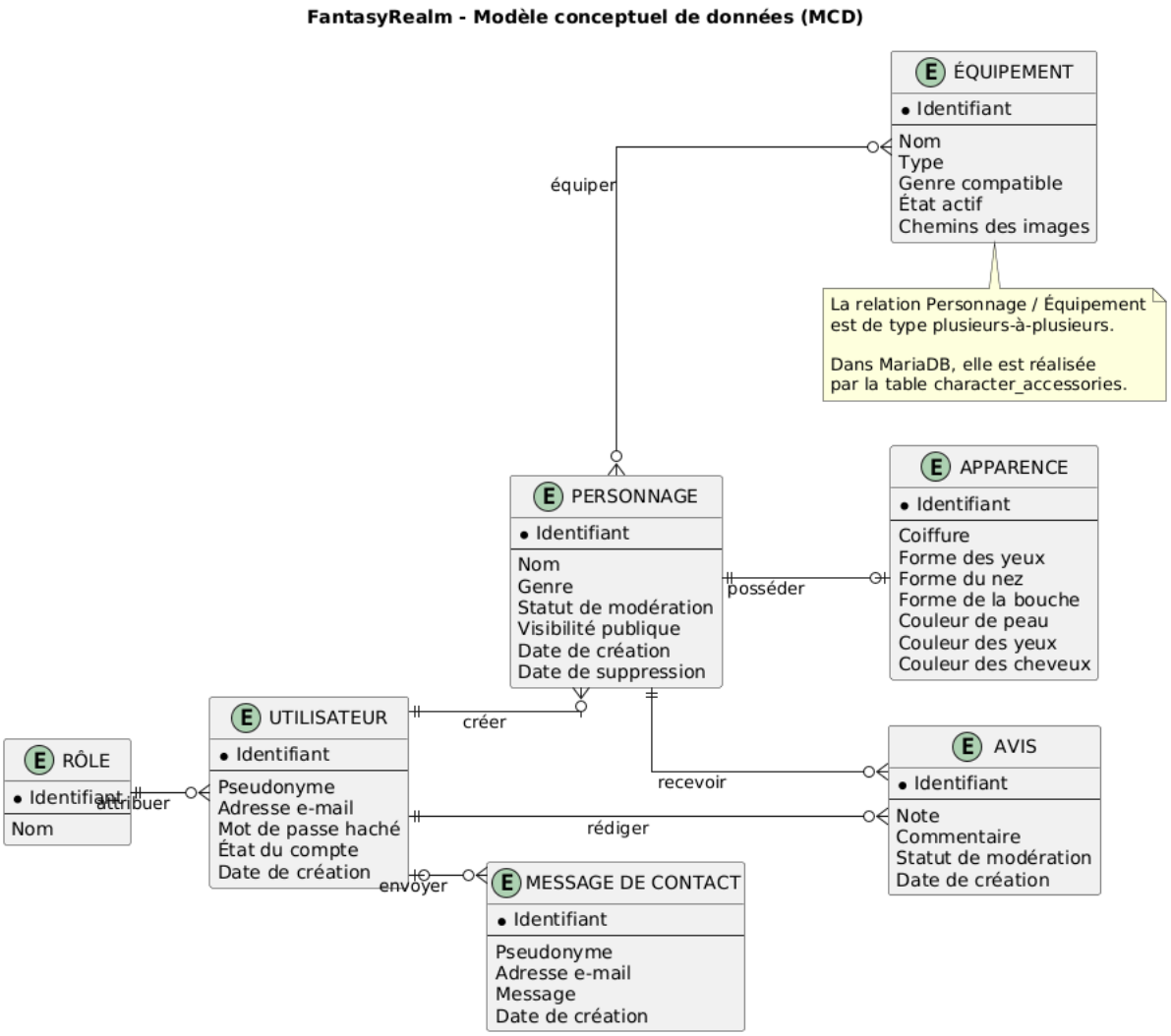
Apache limite les répertoires accessibles par HTTP.

Le reverse proxy prévu pour la production devra assurer la terminaison HTTPS et transmettre correctement les informations de protocole à l'application.

Les en-têtes de sécurité, les politiques de conservation des données, les permissions sur les fichiers et les protections contre les abus devront faire partie des vérifications de production.

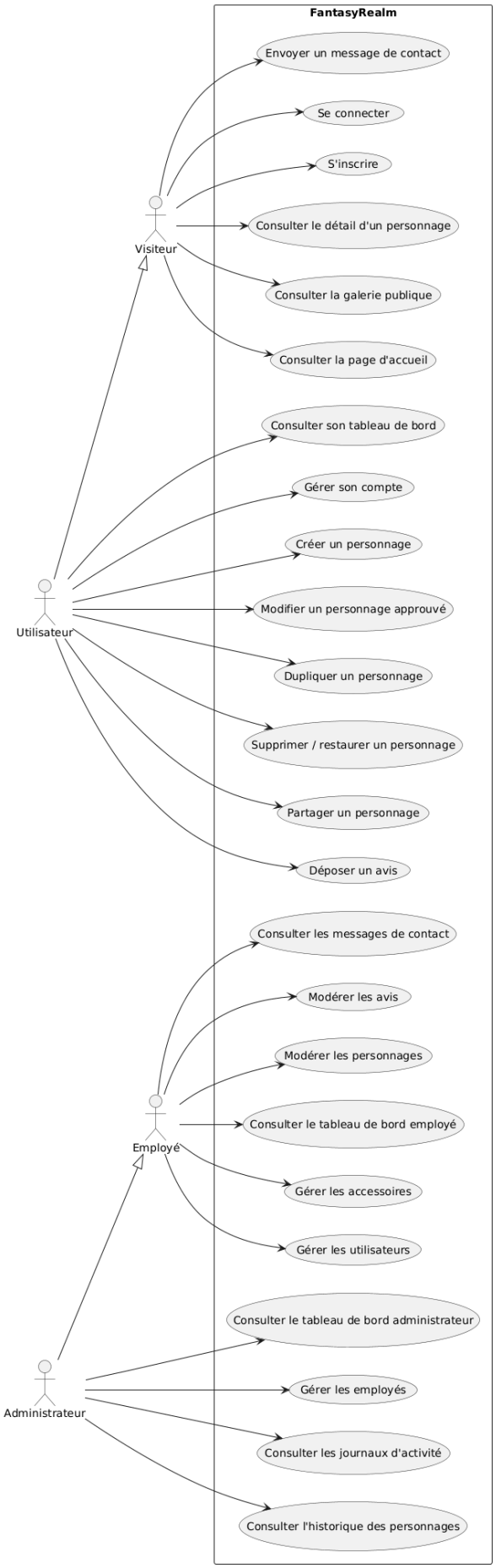
6. DIAGRAMMES TECHNIQUES

6.1. Modèle conceptuel de données (MCD)



Ce diagramme représente les principales entités de la base de données relationnelle FantasyRealm ainsi que les relations qui les unissent. Il permet de visualiser l'organisation des utilisateurs, des personnages, de leurs apparences, des équipements et des avis. Les relations et les cardinalités facilitent la compréhension de la structure des données et garantissent leur cohérence.

6.2. Diagramme de cas d'utilisation UML

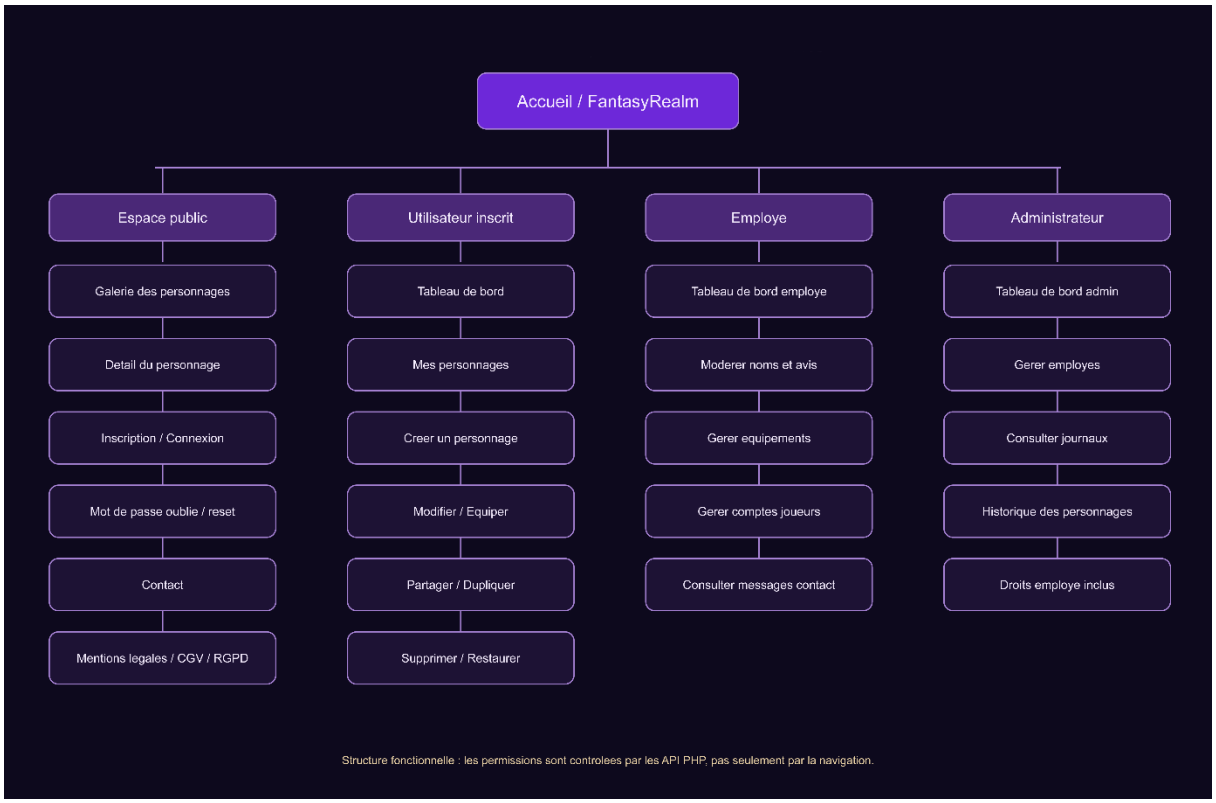


Ce diagramme de cas d'utilisation présente les principales fonctionnalités de FantasyRealm selon les différents profils : visiteur, utilisateur inscrit, employé et administrateur.

Les relations de généralisation indiquent que l'utilisateur inscrit bénéficie également des fonctionnalités publiques du visiteur, tandis que l'administrateur hérite des fonctionnalités de l'employé.

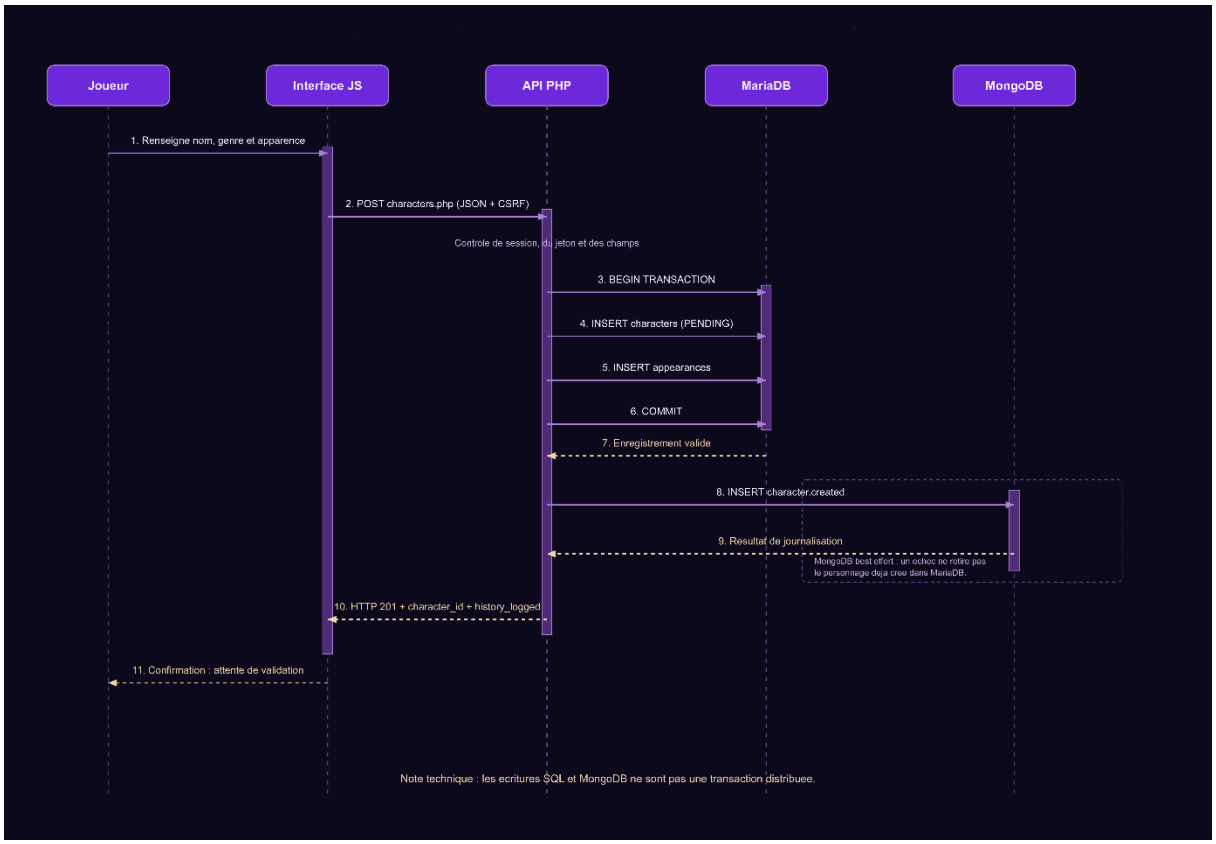
Le diagramme permet ainsi de visualiser les interactions possibles avec l'application et de distinguer les responsabilités des différents acteurs.

6.3. Diagramme de séquence UML — Création d'un personnage



Ce diagramme illustre les échanges réalisés lors de la création d'un personnage. Après la saisie des informations par l'utilisateur, JavaScript transmet les données à l'API PHP, qui effectue les vérifications nécessaires avant leur enregistrement dans MariaDB. Une fois la transaction SQL validée, un historique de création est enregistré dans MongoDB lorsque le service est disponible. Le serveur retourne ensuite une réponse permettant d'informer l'utilisateur du résultat.

6.4. Arborescence du site



Cette arborescence présente l'organisation des différentes pages de FantasyRealm et leur répartition selon les profils utilisateurs. Elle distingue les pages publiques des espaces réservés aux joueurs, aux employés et aux administrateurs. Elle permet de comprendre la structure de navigation et la manière dont les fonctionnalités sont organisées dans l'application.

7. TESTS, MAINTENANCE ET ÉVOLUTIONS

7.1. Tests fonctionnels

Les parcours essentiels doivent être vérifiés afin de confirmer le respect des règles métier.

Les principaux scénarios concernent :

- La création et l'authentification d'un utilisateur.
- Le rejet des doublons d'adresse électronique et de pseudonyme.
- La création d'un personnage avec une apparence valide.
- Le rejet des valeurs d'apparence non autorisées.
- L'approbation et le refus d'un personnage.

- La modification d'un personnage appartenant à l'utilisateur.
- Le contrôle des équipements incompatibles.
- La duplication d'un personnage.
- Le partage et l'arrêt du partage.
- La suppression et la restauration.
- La création et la modération des avis.
- La gestion des comptes et des équipements.
- La consultation des journaux.
- L'envoi des notifications électroniques.
- L'enregistrement des messages de contact.

La vérification doit également comprendre des scénarios d'accès non autorisé et des tests avec des comptes suspendus.

7.2. Tests techniques

Les contrôles techniques porteront notamment sur :

- La construction de l'image Docker.
- Le démarrage des trois services.
- La santé de MariaDB.
- La communication PHP/MariaDB et PHP/MongoDB.
- La persistance des données.
- La disponibilité des ressources graphiques.
- Les performances du moteur Canvas.
- La validité des réponses JSON.
- Les erreurs d'accès aux bases.
- Les scénarios de défaillance de MongoDB.
- Les problèmes de connexion SMTP.

Des tests d'accessibilité et d'affichage sur plusieurs résolutions devront compléter la recette.

7.3. Maintenance

Les fichiers Docker Compose et le Dockerfile permettent de faire évoluer l'environnement.

Les dépendances PHP sont gérées avec Composer et le fichier de verrouillage composer.lock.

Le versionnement Git permet de conserver l'historique des modifications applicatives.

Les données MariaDB et MongoDB sont stockées dans des volumes persistants.

Des procédures de sauvegarde, de restauration et de mise à jour devront être définies pour la production.

Les modifications du schéma SQL devront être réalisées sans écraser les données existantes.

7.4. Évolutions possibles

Plusieurs améliorations pourront être envisagées :

- Enrichissement des options de personnalisation.
- Ajout de nouvelles catégories d'équipements.
- Optimisation du chargement des images.
- Amélioration des outils de recherche dans les galeries.
- Renforcement de la journalisation et de la réconciliation SQL/NoSQL.
- Mise en place de tests automatisés supplémentaires.
- Amélioration continue de l'accessibilité.
- Optimisation des performances et des sauvegardes.
- Internationalisation de l'interface.

Ces évolutions devront être intégrées en conservant la séparation des responsabilités entre frontend, backend et bases de données.

CONCLUSION

FantasyRealm repose sur une architecture web permettant de séparer les interfaces utilisateur, les traitements serveur et les systèmes de stockage.

Le frontend utilise HTML, CSS et JavaScript natif, avec un moteur Canvas pour la composition graphique des personnages.

Le backend PHP assure les opérations métier, l'authentification, les autorisations, la validation des données et les communications avec les bases de données.

MariaDB garantit la cohérence des données principales, tandis que MongoDB permet de conserver les activités et les historiques de personnalisation.

Les contrôles de sécurité intégrés comprennent notamment le hachage Argon2id, les sessions sécurisées, les protections CSRF, les requêtes préparées et les restrictions d'accès selon les rôles.

Docker Compose permet d'exécuter l'ensemble des composants dans un environnement reproductible.

La conception retenue fournit une base pour la maintenance, l'administration et les évolutions futures de FantasyRealm.